- CustomerApiApplication.java:

```java
@SpringBootApplication
public class CustomerApiApplication {

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        SpringApplication.run(CustomerApiApplication.class, args);
    }

}
```

1. Bootstraps Spring Boot application.
2. Scans components, entities, repositories and starts embedded server.
3. Delegates runtime lifecycle to Spring context.

- CustomerServiceImpl.java:

```java
@Service
@Transactional
public class CustomerServiceImpl implements CustomerService {

    private final CustomerRepository customerRepository;

    @Autowired
    public CustomerServiceImpl(CustomerRepository customerRepository) {
        this.customerRepository = customerRepository;
    }

    @Override
    public List<CustomerResponseDTO> getAllCustomers() {
        return customerRepository.findAll()
                .stream()
                .map(this::convertToResponseDTO)
                .collect(Collectors.toList());
    }

    @Override
    public CustomerResponseDTO getCustomerById(Long id) {
        Customer customer = customerRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Customer not found with id: " + id));
        return convertToResponseDTO(customer);
    }

    @Override
    public CustomerResponseDTO createCustomer(CustomerRequestDTO requestDTO) {
        // Check for duplicates
        if (customerRepository.existsByCustomerCode(requestDTO.getCustomerCode())) {
            throw new DuplicateResourceException("Customer code already exists: " + requestDTO.getCustomerCode());
        }

        if (customerRepository.existsByEmail(requestDTO.getEmail())) {
```

1. Implements CustomerService and contains business logic for CRUD, search, filtering, pagination and sorting.

2. Validates duplicates (customerCode, email) and throws domain exceptions.
3. Converts between DTOs and Customer entity for persistence and responses.
4. Uses repository for DB operations and applies transactional boundaries.
5. Implements partial update logic with uniqueness checks.

- CustomerService.java (interface):

```java
public interface CustomerService {
    List<CustomerResponseDTO> getAllCustomers();
    CustomerResponseDTO getCustomerById(Long id);
    CustomerResponseDTO createCustomer(CustomerRequestDTO requestDTO);
    CustomerResponseDTO partialUpdateCustomer(Long id, CustomerUpdateDTO updateDTO);
    Page<CustomerResponseDTO> getAllCustomers(int page, int size, String sortBy, String sortDir);
    // ...other methods...
}
```

1. Declares service API: list, get, create, update, partial update, delete, search, filtering and pagination.
2. Serves as contract for service implementations.

- CustomerRepository.java:

```java
public interface CustomerRepository extends JpaRepository<Customer, Long> {

    Optional<Customer> findByCustomerCode(String customerCode);

    Optional<Customer> findByEmail(String email);

    boolean existsByCustomerCode(String customerCode);

    boolean existsByEmail(String email);

    List<Customer> findByStatus(CustomerStatus status);

    @Query("SELECT c FROM Customer c WHERE " +
           "LOWER(c.fullName) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
           "LOWER(c.email) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
           "LOWER(c.customerCode) LIKE LOWER(CONCAT('%', :keyword, '%'))")
    List<Customer> searchCustomers(@Param("keyword") String keyword);
}
```

1. Extends JpaRepository to provide CRUD and pageable DB access for Customer.
2. Declares finder methods (by code, email, status) and existence checks.

3. Provides a JPQL @Query searchCustomers for keyword searching across fields.

- ResourceNotFoundException.java:

```java
public class ResourceNotFoundException extends RuntimeException {

    public ResourceNotFoundException(String message) {
        super(message);
    }

    public ResourceNotFoundException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

1. Custom RuntimeException used when a requested resource is missing (maps to 404 in handlers).

- DuplicateResourceException.java:

```java
public class DuplicateResourceException extends RuntimeException {

    public DuplicateResourceException(String message) {
        super(message);
    }

    public DuplicateResourceException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

1. Custom RuntimeException thrown when a unique constraint would be violated (maps to 409 in handlers).

- GlobalExceptionHandler.java:

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    // Handle ResourceNotFoundException (404)
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponseDTO> handleResourceNotFoundException(
            ResourceNotFoundException ex,
            WebRequest request) {

        ErrorResponseDTO error = new ErrorResponseDTO(
            HttpStatus.NOT_FOUND.value(),
            error: "Not Found",
            ex.getMessage(),
            request.getDescription(includeClientInfo: false).replace(target: "uri=", replacement: "")
        );

        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    // Handle DuplicateResourceException (409)
    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ErrorResponseDTO> handleDuplicateResourceException(
            DuplicateResourceException ex,
            WebRequest request) {

        ErrorResponseDTO error = new ErrorResponseDTO(
            HttpStatus.CONFLICT.value(),
            error: "Conflict",
            ex.getMessage(),
            request.getDescription(includeClientInfo: false).replace(target: "uri=", replacement: "")
        );

        return new ResponseEntity<>(error, HttpStatus.CONFLICT);
    }

    // Handle Validation Errors (400)
```

1. Catches domain and framework exceptions across controllers (@RestControllerAdvice).
2. Maps exceptions to ErrorResponseDTO with appropriate HTTP status: 404, 409, 400 and 500.
3. Aggregates validation field errors into details for 400 responses.

- CustomerStatus.java (enum):

```java
public enum CustomerStatus {
    ACTIVE,
    INACTIVE
}
```

1. Defines allowed customer states: ACTIVE and INACTIVE.
2. Used by entity, repository and service filtering/validation.

- Customer.java (entity):

```java
@Entity
@Table(name = "customers")
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "customer_code", unique = true, nullable = false, length = 20)
    private String customerCode;

    @Column(name = "full_name", nullable = false, length = 100)
    private String fullName;

    @Column(unique = true, nullable = false, length = 100)
    private String email;

    @Column(length = 20)
    private String phone;

    @Column(columnDefinition = "TEXT")
    private String address;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private CustomerStatus status = CustomerStatus.ACTIVE;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        this.updatedAt = LocalDateTime.now();
    }
```

1. JPA entity mapping customers table with fields (id, code, name, email, phone, address, status, timestamps).
2. Enforces unique/nullable constraints at column level.
3. Maintains createdAt/updatedAt via @PrePersist/@PreUpdate lifecycle callbacks.

- ErrorResponseDTO.java:

```java
public class ErrorResponseDTO {

    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private String path;
    private List<String> details;

    public ErrorResponseDTO() {
        this.timestamp = LocalDateTime.now();
    }

    public ErrorResponseDTO(int status, String error, String message, String path) {
        this.timestamp = LocalDateTime.now();
        this.status = status;
        this.error = error;
        this.message = message;
        this.path = path;
    }

    // Getters and Setters
    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(LocalDateTime timestamp) {
        this.timestamp = timestamp;
    }

    public int getStatus() {
        return status;
    }

    public void setStatus(int status) {
        this.status = status;
    }

    public String getError() {
        return error;
    }

    public void setError(String error) {
```

1. DTO used by GlobalExceptionHandler to return structured error responses.

2. Contains timestamp, status, error, message, path and optional details list.

- CustomerUpdateDTO.java:

```java
public class CustomerUpdateDTO {
    private String fullName;
    private String email;
    private String phone;
    private String address;

    public CustomerUpdateDTO() {}

    public String getFullName() {
        return fullName;
    }
    public void setFullName(String fullName) {
        this.fullName = fullName;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String getPhone() {
        return phone;
    }
    public void setPhone(String phone) {
        this.phone = phone;
    }
    public String getAddress() {
        return address;
    }
    public void setAddress(String address) {
        this.address = address;
    }
}
```

1. Lightweight DTO for PATCH partial updates; fields are optional.
2. Passed to service.partialUpdateCustomer to update only supplied fields.

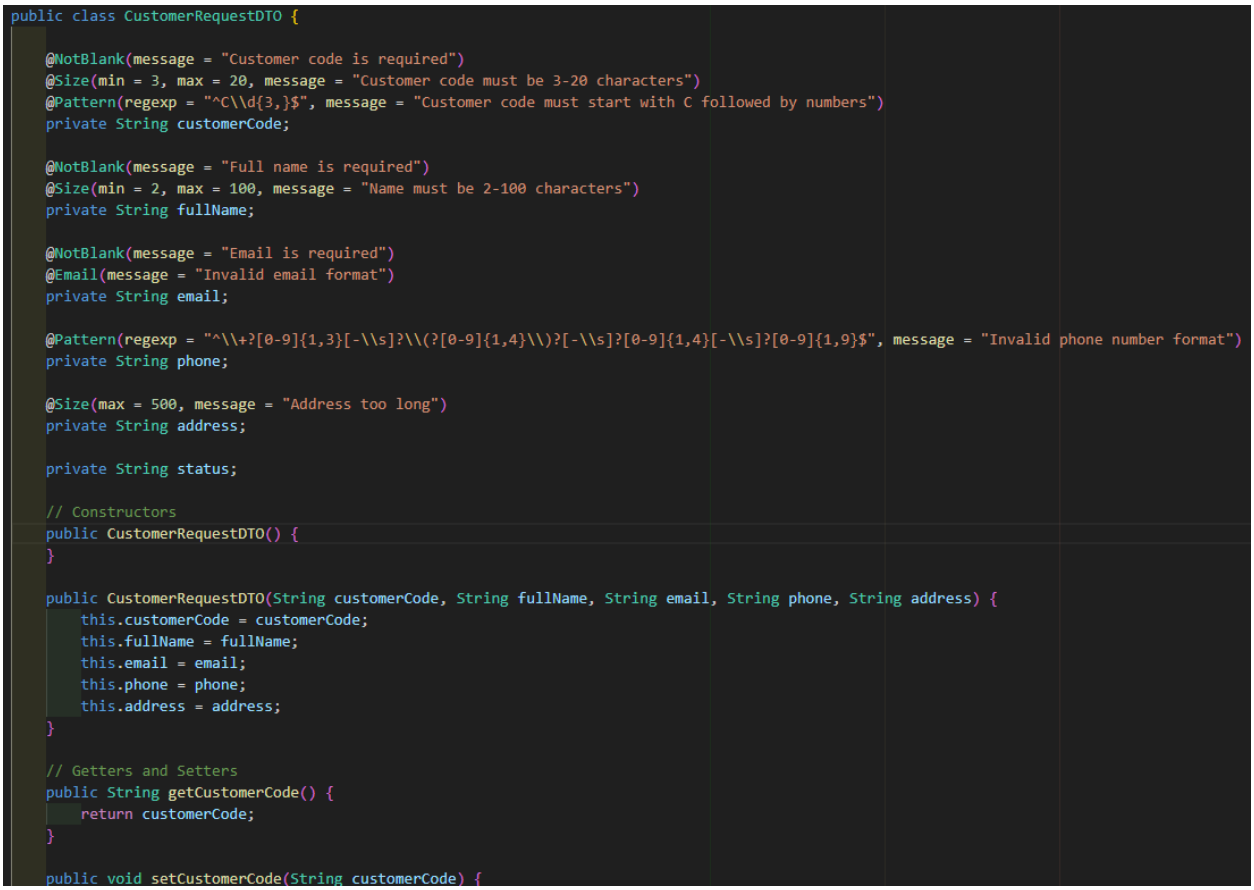
- CustomerResponseDTO.java:

```java
public CustomerResponseDTO() {
}

public CustomerResponseDTO(Long id, String customerCode, String fullName, String email,
    String phone, String address, String status, LocalDateTime createdAt) {
    this.id = id;
    this.customerCode = customerCode;
    this.fullName = fullName;
    this.email = email;
    this.phone = phone;
    this.address = address;
    this.status = status;
    this.createdAt = createdAt;
}

// Getters and Setters
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getCustomerCode() {
    return customerCode;
}

public void setCustomerCode(String customerCode) {
    this.customerCode = customerCode;
}

public String getFullName() {
    return fullName;
}

public void setFullName(String fullName) {
    this.fullName = fullName;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
```

1. DTO returned to clients for read operations.

2. Exposes id, customerCode, fullName, email, phone, address, status and createdAt.

- CustomerRequestDTO.java:



```java
public class CustomerRequestDTO {

    @NotBlank(message = "Customer code is required")
    @Size(min = 3, max = 20, message = "Customer code must be 3-20 characters")
    @Pattern(regexp = "^C\\d{3,}$", message = "Customer code must start with C followed by numbers")
    private String customerCode;

    @NotBlank(message = "Full name is required")
    @Size(min = 2, max = 100, message = "Name must be 2-100 characters")
    private String fullName;

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;

    @Pattern(regexp = "^\\+?[0-9]{1,3}[-\\s]?\\(?[0-9]{1,4}\\)?[-\\s]?[0-9]{1,4}[-\\s]?[0-9]{1,9}$", message = "Invalid phone number format")
    private String phone;

    @Size(max = 500, message = "Address too long")
    private String address;

    private String status;

    // Constructors
    public CustomerRequestDTO() {
    }

    public CustomerRequestDTO(String customerCode, String fullName, String email, String phone, String address) {
        this.customerCode = customerCode;
        this.fullName = fullName;
        this.email = email;
        this.phone = phone;
        this.address = address;
    }

    // Getters and Setters
    public String getCustomerCode() {
        return customerCode;
    }

    public void setCustomerCode(String customerCode) {
```

1. DTO used for create (and full update) requests with validation annotations (@NotBlank, @Email, @Pattern, @Size).
2. Carries customerCode, fullName, email, phone, address and optional status.

- CustomerRestController.java:

```java
@RestController
@RequestMapping("/api/customers")
@CrossOrigin(origins = "*")
public class CustomerRestController {

    private final CustomerService customerService;

    @Autowired
    public CustomerRestController(CustomerService customerService) {
        this.customerService = customerService;
    }

    // GET all customers (supports optional sorting)
    @GetMapping
    public ResponseEntity<List<CustomerResponseDTO>> getAllCustomers(
            @RequestParam(required = false) String sortBy,
            @RequestParam(defaultValue = "asc") String sortDir) {
        if (sortBy == null || sortBy.trim().isEmpty()) {
            List<CustomerResponseDTO> customers = customerService.getAllCustomers();
            return ResponseEntity.ok(customers);
        }
        List<CustomerResponseDTO> customers = customerService.getAllCustomers(sortBy, sortDir);
        return ResponseEntity.ok(customers);
    }

    // GET search customers (more specific, before /{id})
    @GetMapping("/search")
    public ResponseEntity<List<CustomerResponseDTO>> searchCustomers(@RequestParam String keyword) {
        List<CustomerResponseDTO> customers = customerService.searchCustomers(keyword);
        return ResponseEntity.ok(customers);
    }

    // GET advanced search customers
    @GetMapping("/advanced-search")
    public ResponseEntity<List<CustomerResponseDTO>> advancedSearch(
        @RequestParam(required = false) String name,
        @RequestParam(required = false) String email,
        @RequestParam(required = false) String status) {
        List<CustomerResponseDTO> customers = customerService.advancedSearch(name, email, status);
        return ResponseEntity.ok(customers);
    }

    // GET customers by status (more specific, before /{id})
    @GetMapping("/status/{status}")
    public ResponseEntity<List<CustomerResponseDTO>> getCustomersByStatus(@PathVariable String status) {
```

1. Exposes REST endpoints under /api/customers for list/search/filter/page/get/create/update/patch/delete.
2. Validates incoming DTOs (@Valid), delegates to CustomerService, and returns appropriate ResponseEntity payloads and statuses.
3. Orders specific routes (search/status/page) before the generic /{id} path to avoid routing conflicts.